

Лабораторная работа 2. Создание смарт-контракта и взаимодействие с ним

Цель

Познакомиться с языком программирования смарт-контрактов Solidity, познакомиться с методами взаимодействия со смарт-контрактами из dApp.

Шаги:

1. Создать смарт-контракт
2. Развернуть его на блокчейне
3. Создать пользовательский интерфейс для взаимодействия со смарт-контрактом

Замечание об окружении

Ниже подразумевается, что вы работаете в UNIX-подобной системе: Linux, FreeBSD, MacOS, и т.п. Если вы работаете с Microsoft Windows, вам потребуется дополнительно установить Linux или использовать виртуальную машину с Linux через Windows Subsystem for Vagrant, VirtualBox и др.

Примечание: В данной работе был использован ubuntu server

```
bitcoin@bitcoin:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.3 LTS
Release:        24.04
Codename:       noble
bitcoin@bitcoin:~$
```

Создание смарт-контракта

Для выполнения лабораторной работы предлагается использовать набор инструментов разработки Truffle. Для его выполнения требуется установить на компьютер последнюю LTS версию Node.js и NPM (поставляется вместе с Node.js) с официального сайта: <https://nodejs.org/ru>

Download and install nvm:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash
```

in lieu of restarting the shell

```
\. "$HOME/.nvm/nvm.sh"
```

Download and install Node.js:

```
nvm install 20
```

Verify the Node.js version:

```
node -v # Should print "v22.20.0".
```

Verify npm version:

`npm -v` # Should print "10.9.3".

Когда Node.js установлена, следует установить truffle:

Вначале установите необходимые пакеты:

`$ sudo apt install build-essential`

Для установки Truffle введите команду:

`$ npm install -g truffle`

После установки проверьте версию Truffle:

`$ truffle version`

С помощью Ganache CLI можно очень легко создать персональный блокчейн, удобный для отладки смарт-контрактов, в том числе с помощью такого инструмента, как Truffle.

Установить Ganache CLI очень просто:

`$ sudo npm install -g ganache-cli`

Truffle создаёт проект в текущей папке. Создадим новую папку для контракта, и перейдём в неё:

`$ mkdir poster-contract`

`$ cd poster-contract`

Теперь в соседнем терминале, в той же папке poster-contract запустим локальный узел Ganache:

`$ ganache-cli`

В консоль Ganache выведет диагностическую информацию:

```

bitcoin@bitcoin:~/poster-contract$ ganache-cli
Ganache CLI v6.12.2 (ganache-core: 2.13.2)

Available Accounts
=====
(0) 0x02E6C641487856f5cf776c6313c72fedFC4fb980 (100 ETH)
(1) 0x921933898f8AdD10A24bd6C2C1D41a56F87202AF (100 ETH)
(2) 0xae00F10b7d5958C6FCFDE8932320dBf84eE34f5f (100 ETH)
(3) 0xc5d997a693f40628d7F63b3C75592a9C4e3Fd717 (100 ETH)
(4) 0x033F0E5b7B7f4b7536f27480F36B0D7a86C8C35A (100 ETH)
(5) 0x6f6536E5f71A2a14EE6bAC92Dcc045a3099d3CBc (100 ETH)
(6) 0x28b2Ea4268109c45F6b02cdf078508F8b91D166B (100 ETH)
(7) 0x1051800C2C58dcE6f14091E186A849a2f1C5e4eD (100 ETH)
(8) 0x5860D2F9BC9a32908799f7a403405743A6Ad3Ad9 (100 ETH)
(9) 0x741aB20C8Ef68960404B4D821eFC2d8c81cCE50 (100 ETH)

Private Keys
=====
(0) 0xc20a478f2b5735c6ec73ccbde1c82902301e2458185cbaceaa20cf3f5675318a
(1) 0xd29b87e3ad4a172960785e09dd35b32e0b9a6e8c9195a38ab5182e6e02c0c87b
(2) 0x769018b632e01b8aff6e3e567d007ec4b5086efcacf0f8cba04cba3a4bd0044c
(3) 0xfd26bfab8375f0e6377902ab908e9b095c26ec64683922eeb1935432c64746d1
(4) 0xa876b71fb034ec667a2f68d3a7570ca21ebfa29c1ed33f4ae24d7fd31e990f2e
(5) 0x3b62ffd255dad501b411c9abdbfb9d419a90bb02d001fbd9399c34bb4336
(6) 0xfcfc5827e7ab91eb1f97c0847c777a36d23e1bf480befaa9abd5b6a829c574a95
(7) 0x751e460fb1c10c5b02052c349a2c222ddaa3304aaad28a941920a20cfd880a57
(8) 0x25bab91a34c91615eac7590896219b5775d843750bc7bfa483f679095ed9c088
(9) 0x2a7e5350e4220dd362869c2d71ba77a113f3285840b45966dc6d4c7220cc577d

HD Wallet
=====
Mnemonic:      beach three immune barely win toast ripple dragon swing taste summer cube
Base HD Path:  m/44'/60'/0'/0/{account_index}

Gas Price
=====
20000000000

Gas Limit
=====
6721975

Call Gas Limit
=====
9007199254740991

Listening on 127.0.0.1:8545

```

Воспользуемся в основной консоли уже существующим шаблоном (или Truffle Box) для создания контрактов на сети Polygon:

```
$ cd poster-contract
```

```
$ truffle unbox polygon
```

Команда создаст в текущей папке минимальный набор файлов для начала работы:

- contracts - содержит код смарт-контрактов
 - ethereum - код смарт-контрактов для Ethereum (нам не потребуется)
 - polygon - код смарт-контрактов для сети Polygon
- migrations - набор файлов для разворачивания и/или изменения разрабатываемых смарт контрактов
- test - содержит тесты. Чтобы проверить корректность работы, запустим тесты:

Чтобы проверить корректность работы, запустим тесты:

```
$ npm run test
```

```
bitcoin@bitcoin:~/poster-contract$ npm run test
> polygon-box@1.0.0 test
> truffle test

Compiling your contracts...
=====
> Compiling ./contracts/ethereum/SimpleStorage.sol
> Artifacts written to /tmp/test--3629-lJ3i6yviul8S
> Compiled successfully using:
   - solc: 0.5.16+commit.9c3226ce.Emscripten.clang

Contract: SimpleStorage
  ✓ ...should store the value 89. (243ms)

1 passing (289ms)
bitcoin@bitcoin:~/poster-contract$ |
```

Теперь время создавать свой смарт-контракт. К несчастью, используемый нами шаблон содержит некоторое количество мусора. Давайте его выкинем. Для этого, перенесите имеющийся контракт SimpleStorage в папку contracts и удалите ненужные папки внутри contracts:

```
$ mv ./contracts/polygon/SimpleStorage.sol ./contracts
```

```
$ rm -rf ./contracts/ethereum ./contracts/polygon
```

Удалите truffle-config.js и переименуйте truffle-config.polygon.js на truffle-config.js :

```
$ rm ./truffle-config.js
```

```
$ mv ./truffle-config.polygon.js ./truffle-config.js
```

Внутри конфигурационного файла ./truffle-config.js вы можете увидеть, что он использует ссылки на Polygon-специфичные папки. Давайте заменим их на значения по умолчанию, удалив поля:

contracts_build_directory : значение по умолчанию ./build/contracts ,

contracts_directory : значение по умолчанию ./contracts .

Кроме того, имеет смысл переименовать названия сетей:

polygon_infura_testnet → polygon_mumbai ,

polygon_infura_mainnet → polygon_matic

Примечание: Mumbai больше не поддерживается, вместо него будет использоваться Ethereum. Но так как polygon_mumbai и polygon_matic это переменные – оставим как есть.

Дополнительно следует убрать ссылки на Polygon-специфичный конфигурационный файл из package.json . Уберите все строки вида --config=truffle-config.polygon.js и переименуйте элементы в поле scripts так, чтобы они не ссылались на Polygon. Должно получиться такое содержание поля scripts:

```
"scripts": {
  "test": "truffle test",
  "test": "truffle test --network=$npm_config_network",
```

```
"compile": "truffle compile",  
"migrate": "truffle migrate --network=$npm_config_network"  
},
```

Теперь время создать свой контракт гостевой книги Poster. Создайте файл Poster.sol в папке contracts . Он должен иметь вид:

```
// SPDX-License-Identifier: MIT  
pragma solidity >=0.4.21 <0.7.0;  
  
contract Poster {  
    event NewPost(address indexed user, string content, string indexed tag);  
  
    function post(string memory content, string memory tag) public {  
        emit NewPost(msg.sender, content, tag);  
    }  
}
```

Единственная функция, которая здесь выполняется, это post . Её задача - принять пользовательские параметры и положить как событие на блокчейн. Создадим тест для этой функции в файле test/poster.js по аналогии с существующим тестом test/simplestorage.js :

```
const Poster = artifacts.require("./Poster.sol");  
  
contract("Poster", accounts => {  
    it("emit event", async () => {  
        const posterInstance = await Poster.deployed();  
        const eventsBefore = await posterInstance.getPastEvents('NewPost')  
        assert.deepEqual(eventsBefore, [])  
        const content = "Hello, world!"  
        const tag = "hello"  
        await posterInstance.post(content, tag, {from: accounts[0]})  
        const eventsAfter = await posterInstance.getPastEvents('NewPost')  
        assert.equal(eventsAfter.length, 1)  
        const postedEvent = eventsAfter[0]  
        assert.equal(postedEvent.args.user, accounts[0])  
        assert.equal(postedEvent.args.content, content)
```

```

    assert.equal(postedEvent.args.tag, web3.utils.keccak256(tag))
  });
});

```

Создадим файл migrations/2_poster.js со следующим содержанием:

```
const Poster = artifacts.require("Poster");
```

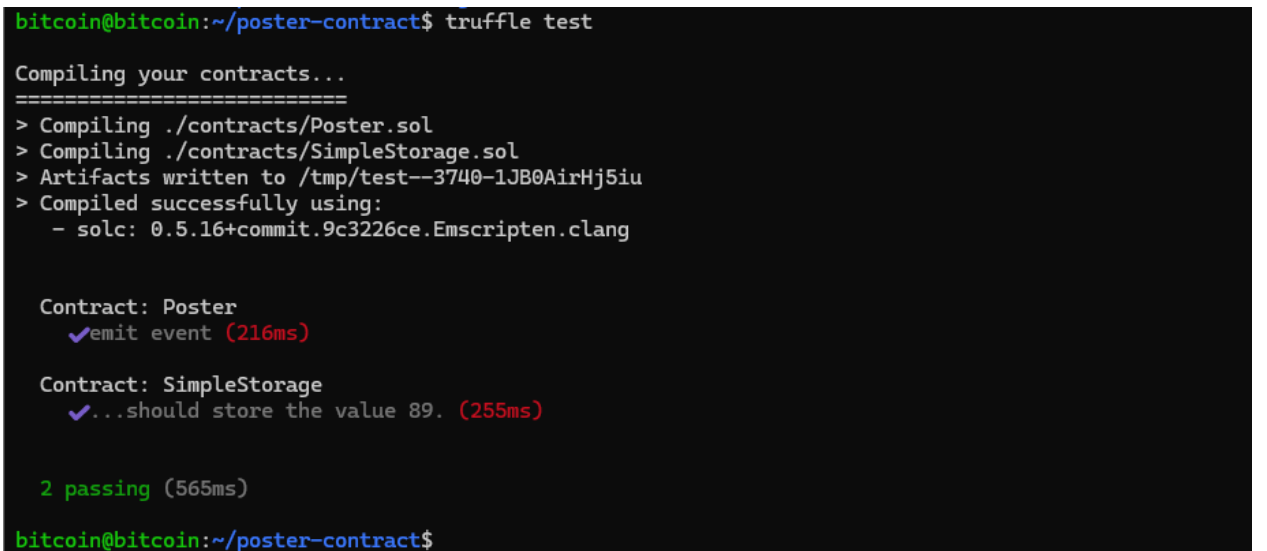
```

module.exports = function (deployer) {
  deployer.deploy(Poster);
};

```

Теперь запустим тесты через truffle test . Команда выдаст положительный результат тестирования:

```
$ truffle test
```



```

bitcoin@bitcoin:~/poster-contract$ truffle test

Compiling your contracts...
=====
> Compiling ./contracts/Poster.sol
> Compiling ./contracts/SimpleStorage.sol
> Artifacts written to /tmp/test--3740-1JB0AirHj5iu
> Compiled successfully using:
   - solc: 0.5.16+commit.9c3226ce.Emscripten.clang

Contract: Poster
  ✓emit event (216ms)

Contract: SimpleStorage
  ✓...should store the value 89. (255ms)

2 passing (565ms)

bitcoin@bitcoin:~/poster-contract$

```

Теперь, когда мы имеем протестированный смарт-контракт, имеет смысл его развернуть на блокчейн.

2. Развёртывание контракта на блокчейн

Для развёртывания контракта на блокчейне нужны следующие элементы:

байткод контракта,

доступ к сети, реализуемый через Ethereum JSON RPC URL,

Эфир (или Matic, или другой нативный токен), чтобы заплатить за транзакцию создания контракта на блокчейне,

Байт-код контракта Truffle создаёт по команде truffle compile . После выполнения этой команды вы можете заметить появление новых файлов в папке build/contracts . Каждый из файл там содержит помимо прочего байткод контракта и его ABI.

Выполните

```
$ truffle compile
```

Для доступа к сети нам потребуется Ethereum JSON RPC URL. Мы можем использовать тот же URL, который мы использовали для подключения Метамаска к Polygon Mumbai. Например, мы можем взять его с ChainList. Мы используем этот URL в конфигурационном файле `truffle-config.js`. Там найдите секцию `polygon_mumbai`. Внутри секции задайте параметр `providerOrUrl` у конструктора `HDWalletProvider` значением вашего JSON RPC URL. Секция `polygon_mumbai` будет выглядеть примерно так:

```
//polygon Infura testnet

polygon_mumbai: {
  provider: () => new HDWalletProvider({
    mnemonic: {
      phrase: mnemonic
    },
    providerOrUrl:
      "https://ethereum-sepolia-rpc.publicnode.com"
  }),
  network_id: 11155111,
  confirmations: 2,
  timeoutBlocks: 200,
  skipDryRun: true,
  chainId: 11155111
}
```

Здесь вы можете обратить внимание на переменную `mnemonic`. Мнемоника – другое название секретной фразы или сид-фразы.

Для оплаты транзакции создания контракта на блокчейне следует передать мнемонику как переменную окружения при запуске `truffle deploy`. Вы можете использовать ту же секретную фразу, которую вы использовали для Метамаска из лабораторной 1. Вы также можете создать новую мнемонику и перевести туда достаточное количество нативных токенов с вашего счёта в Метамаске.

Так, для данной работы можно воспользоваться инструментом `Mnemonic Code Converter` <https://iancoleman.io/bip39/> для создания мнемоники.

Generate a random mnemonic: **GENERATE** 15 words, or enter your own below.

☐ Show entropy details

☐ Hide all private info

☒ Auto compute

Mnemonic Language: English 日本語 Español 中文(简体) 中文(繁體) Français Italiano 한국어 Čeština Português

BIP39 Mnemonic: imitate mirror obscure history sudden dress mango diagram moment mushroom airport faculty rug rail illness

☐ Show split mnemonic cards

BIP39 Passphrase (optional)

BIP39 Seed: 4b0e492bd5fc6a5e0b948da9f09e00dc9aac2d359b352a390c8128dcd9fd4576fae1246c294cabaae8438cf4db88b7c101c2ab835722dcdc1c0c6d9834279cf

Coin: ETH - Ethereum

BIP32 Root Key: xprv9s21ZrQH143K3AbT1ZNwAJBPG4trAQg4RHTEfFkRXMfRz2wfcYxtadwkSUVrBCbjncpufMQ918XxBcQfsrq6kGQ3sZVCf99UKWkfSdnaVjh

☐ Show BIP85

Пролистав страницу этого инструмента вниз вы можете увидеть адреса, выведенные из этой мнемоники. Truffle будет использовать первый из выведенных адресов. Именно сюда вы можете перевести часть нативных токенов с вашего Метамаска.

Derived Addresses

Note these addresses are derived from the BIP32 Extended Key

☐ Encrypt private keys using BIP38 and this password: Enabling BIP38 means each key will take several minutes to generate.

☐ Use hardened addresses

Table CSV

Path	Toggle	Address	Toggle	Public Key	Toggle	Private K
m/44'/60'/0'/0/0		0x1f181B9cE6f98525f0bD152425E6B004269c7671		0x0268781aa32b7b4db53f675bcf1510031850471ce69e637bac2d68ed89e4c6d4c5		0x13bb1
m/44'/60'/0'/0/1		0xA9FA19eFE981b3873671A8f4d5dAe91ac25F135B		0x03a77c36c187fc80e75504939c0ec51dd2b967b7f2e6f577f93607715043c6fd11		0x3711f
m/44'/60'/0'/0/2		0x237d704D2b54dFc3eb59CA3e16Dd1c889eD51FF0		0x0213c45189ce32c7217ad54694ab8d67ad0cd66fe78dc60b99b487cccb7e27a5f7		0xc8c1d
m/44'/60'/0'/0/3		0x4ad77b729D43A2426238eBC258B8fd673Fd9E858		0x02ffa5bbdbb31489af7a3987fb6e97af37a51919679f6bdf8584b4cd4128d34a88		0x5b9ba
m/44'/60'/0'/0/4		0xCbDC3902C021DF73791b307522641d0D62bd2d97		0x02bb6c84c01eeaf8533c8687e2f1024325a6d9fd1ad9abef650bc6c5ced33cb100		0x7799c
m/44'/60'/0'/0/5		0xE7dcc35659b94A672805683cF00EBA7711770f8a		0x0321bb3189628d2b631bfe3f968ffe6c8b57178fdbd75b1710c468c7921ff7da7a		0x364b3
m/44'/60'/0'/0/6		0x80EA18f4B5c920EB8B259eb7CE1d0ecD19b3f062		0x033c88948139cd70e72bdf1a67467c218785a3d584a49f5cedbe1b7e960d24e69d		0x01fba
m/44'/60'/0'/0/7		0xE547F6132AB1369A6C3754460C9856a0B2Bcd5a1		0x0389ed5044beabad70aa29f26bbfdef1cfc3a41bded18ad2dbca74fa7bfade6f59		0xf54f2
m/44'/60'/0'/0/8		0xc86f7068868467BB4738e5593731ca9be9cdf55a		0x02c6a0b795540fdc2394b34364179ed556d2bel110b8896f413a9b60d3de7c2de6		0x95b39
m/44'/60'/0'/0/9		0x62D0753E828B181756876a8f7D2D692e813c0A03		0x03f5b08ff601ffbad73aac600590fc8d1c004d7840b939728a44821c768601f9e		0xf6e51

Следует убедиться, что следующие утверждения справедливы:

- вы можете скомпилировать код контракта в байт-код без ошибок через
- вы передаёте в `truffle compile`, `truffle-config.js` корректный Ethereum JSON RPC URL,
- вы имеете в своём распоряжении корректную секретную фразу,
- соответствующий адрес имеет достаточное TODO количество нативных токенов.

После этого можно запускать команду развёртывания контрактов:

\$ MNEMONIC="here goes your mnemonic phrase" truffle deploy --network=polygon_mumbai

В результате вы должны иметь в консоли что-то подобное:

```
bitcoin@bitcoin:~/poster-contract$ MNEMONIC="gas isolate ... twice" truffle deploy --network=polygon_mumbai

Compiling your contracts...
=====
> Everything is up to date, there is nothing to compile.

Starting migrations...
=====
> Network name:    'polygon_mumbai'
> Network id:     11155111
> Block gas limit: 60000000 (0x3938700)

1_deploy_simple_storage.js
=====

Deploying 'SimpleStorage'
=====
> transaction hash: 0xf434c5953...1b
> Blocks: 1        Seconds: 12
> contract address: 0xC78h...89f
> block number:    93
> block timestamp: 1'...6
> account:         0x30E...1f
> balance:         0.049082738885968942
> gas used:        96205 (0x177cd)
> gas price:       0.001000043 gwei
> value sent:      0 ETH
> total cost:      0.000000096209136815 ETH

Pausing for 2 confirmations...
```

```
> confirmation number: 1 (block: 93)
> confirmation number: 2 (block: 94)
> Saving artifacts
=====
> Total cost:      0.000000096209136815 ETH

2_poster.js
=====

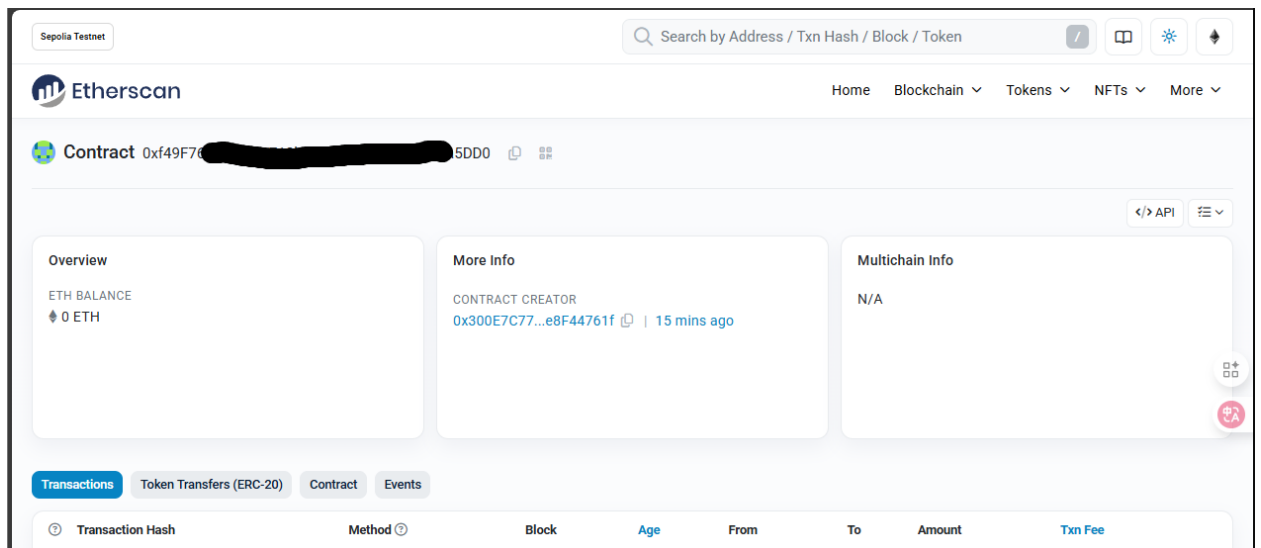
Deploying 'Poster'
=====
> transaction hash: 0xec51d1...2db821
> Blocks: 1        Seconds: 16
> contract address: 0xf49...A3a5DD0
> block number:    94
> block timestamp: 17...4
> account:         0x30E7...44761f
> balance:         0.049082530387212362
> gas used:        208490 (0x32e6a)
> gas price:       0.001000042 gwei
> value sent:      0 ETH
> total cost:      0.00000020849875658 ETH

Pausing for 2 confirmations...

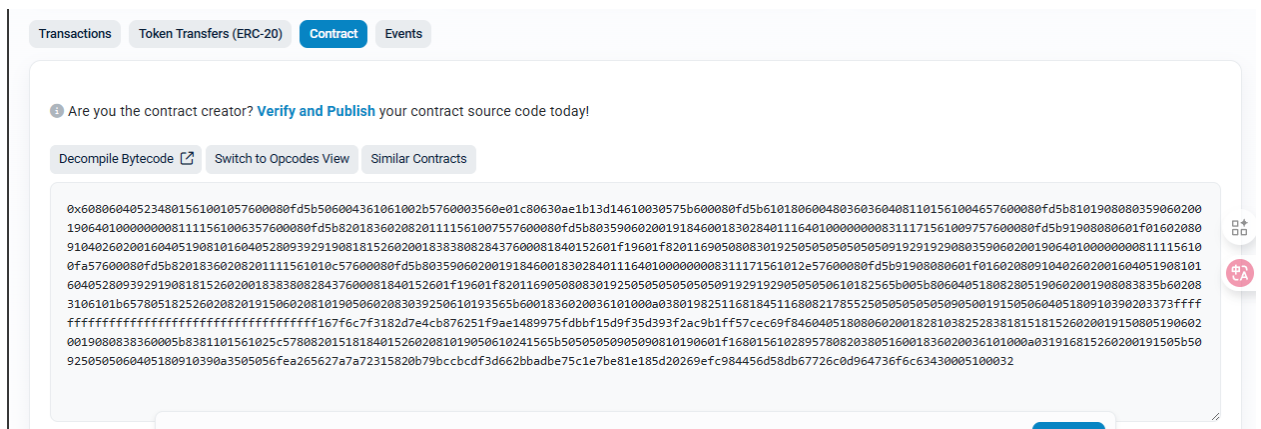
> confirmation number: 1 (block: 93)
> confirmation number: 2 (block: 94)
> Saving artifacts
=====
> Total cost:      0.00000020849875658 ETH
```

```
Summary
=====
> Total deployments: 2
> Final cost:        0.000000304707893395 ETH
```

Вы можете открыть полученный адрес контракта на блокчейн-эксплорере и убедиться, что он в самом деле доступен в сети <https://sepolia.etherscan.io/>.



Если вы перейдёте на вкладку "Contract", вы можете просмотреть байткод контракта.



Правила хорошего тона предписывают раскрытие исходного кода контракта для блокчейн-эксплорера. Это называют "верификацией" смарт-контракта, не путать с формальной верификацией.

Для верификации вам потребуется API-ключ Etherscan. Создайте аккаунт на [Etherscan Client Portal and Services](#) и затем создайте API ключ

Back Home

Account Overview

Sign Out

Account Overview

Account Settings

Public Profile

Lists

Watch List

Private Name Tags

Txn Private Notes

Token Ignore List

Advanced Filter

Others

API Dashboard

Verified Addresses

Custom ABIs

Etherscan Points

Celebrate Etherscan's 10th anniversary by claiming rewards, earning points and levelling up!

View Points

Personal Info

Below are the username, email and overview information for your account.

Your username:

Your Email Address:

Last Login:

2025-09-25 19:08:03 (UTC)

History

Overview Usage

Usage of account features such as address watch list, address name tags, and API keys.

Total ETH Balance (Watch List):

Email Notification Limit:

Address Watch List:

Txn Private Notes

Address Tags:

API Key Usage:

Verified Addresses

0 ETH (\$0.00)

0 emails sent out

0 address alert(s)

0 transaction private note(s)

0 address tag(s)

1 active API(s)

0 verified addresses

100 daily limit

50 limit

10,000 limit

5,000 limit

3 limit

Unlimited

API Keys 3 limits

Add

For developers interested in building applications using our [API Service](#), please create an API-Key Token which you can then use with all your API requests.

App Name	API Key Token	API Statistics	API Logs
2laba	T...P7	Visit Stats	View Logs

1 key used out of 3 available

API keys created on [Etherscan.io](#) can be used across all our [supported chain](#). Detailed documentation to get started can be found at [V2 API Documentation](#). If you need any API support, contact us [here](#).

Теперь вернёмся в папку poster-contract . Добавим плагин truffle-plugin-verify как зависимость:

```
$ npm add truffle-plugin-verify -D
```

Добавим плагин в конфигурационный файл `module.exports = { /* ... предыдущее содержимое */ plugins: ['truffle-plugin-verify'] }`

Добавим плагин в конфигурационный файл `./truffle-config.js`

```
module.exports = {
```

```

/**
 * contracts_build_directory tells Truffle where to store compiled contracts
 */

contracts_build_directory: './build/contracts',
plugins: ['truffle-plugin-verify'],
api_keys: {
  etherscan: 'API_KEY'
},
/* ... следующее содержимое */

```

Теперь вызовем команду верификации. Она тоже требует секретную фразу в переменной MNEMONIC

```
$ MNEMONIC="here goes your mnemonic phrase" truffle run verify Poster --network polygon_mumbai
```

По завершении процесса откройте страницу контракта на экспортере. Заметьте, как изменилось содержимое вкладки "Contract". Теперь можно вызывать функции контракта!

Пользовательский интерфейс для взаимодействия со смарт-контрактом

Теперь наша задача - создать пользовательский интерфейс для работы с контрактом. Можно считать, что это простейший dApp.

Для построения dApp мы будем использовать классический Next.js и React. В родительской для poster-contract папке вызовите команду

```
$ npx create-next-app@latest
```

Это подготовит каркас приложения для пользовательского интерфейса. Команда спросит вас об имени приложения. Выберите poster-ui как имя приложения. По завершении работы команды перейдите в только что созданную папку poster-ui и установите зависимости:

```
$ npm install
```

Теперь можно запускать dev-сервер (`npm run dev`) и открывать основной файл, с которым будем работать: `pages/index.js` .

Он содержит много лишнего, приведем его до состояния продуктивной пустоты:

```

import Head from 'next/head'

import Image from 'next/image'

export default function Home() {
  return (

```

```

    <div>

  </div>

)

}

```

Во-первых, нам необходимо отсюда вызвать функцию `post` . Как указывает лекция, для вызова функции контракта достаточно адреса и ABI контракта. Эти сведения можно узнать или из Truffle artefact в `poster-contract/build/contracts/Poster.json` , либо из блокчейн-эксплорера.

Мы можем использовать библиотеку `web3.js` для взаимодействия с контрактом. Её нужно установить как зависимость:

```
$ npm add web3
```

Одна из сложных часть взаимодействия с контрактами, как это ни удивительно, это аутентификация пользователя в приложении. Вы можете использовать код ниже как стартовую точку. Он запрашивает у пользователя его адрес и позволяет оперировать с ним из приложения.

```

import Head from 'next/head'

import Image from 'next/image'

import {useEffect, useState} from "react";

import Web3 from "web3";

export default function Home() {

  const [web3, setWeb3] = useState(undefined)

  const [userAddress, setUserAddress] = useState(undefined)

  const handleConnect = async () => {

    const web3 = new Web3(window.ethereum)

    const [address] = await window.ethereum.enable()

    setUserAddress(address)

    setWeb3(web3)

  }

  return (

    <div>

      <button onClick={handleConnect}>Connect</button>

      <address>{userAddress}</address>

    </div>

  )

}

```

Для того, чтобы считать, что вы выполнили лабораторную работу, от вас потребуется достроить этот dApp. Необходимо предоставить вашему пользователю:

1. Возможность постить текст в контракт через функцию контракта `post` .
2. Возможность смотреть текст, тег и адрес пользователя для всех, кто запостил сообщение в контракт.
3. Возможность фильтровать посты по тегам

Формат предоставления отчёта

В результате работы у вас должно получиться совсем небольшое, но законченное приложение. Код приложения - контракты и пользовательский интерфейс - должен быть доступен в системе контроля версий: GitHub, GitLab, Radicle и т.д.

По завершении кодирования попросите своих одноклассников воспользоваться вашим приложением.

- Отчёт должен содержать:
- ссылку на верифицированный контракт на блокчейн-эксплорере,
- ссылку на репозиторий,
- описание вашего процесса работы над приложением,
- описание приложения как краткое руководство пользователя